

Introduction to R and RStudio

Sushank Pokhrel (Homepage) (<https://sushank.com.np>)

What is R?

R is a programming language and software environment specifically designed for statistical computing, data analysis, and graphical representation. It is widely used by data scientists, statisticians, and researchers across various domains, including academia, business, and healthcare.

Key Features of R:

- 1. Statistical Analysis: R provides a vast array of statistical techniques such as regression, hypothesis testing, time-series analysis, and machine learning.
- 2. Data Visualization: Tools like `ggplot2` and `lattice` allow users to create high-quality visualizations.
- 3. Community Support: R has a large and active community that contributes to its extensive library of packages available on CRAN (Comprehensive R Archive Network).
- 4. Flexibility and Extensibility: You can create custom functions, packages, and scripts to suit your needs.

R can be run on multiple operating systems, including Windows, macOS, and Linux, making it accessible to a wide audience.

What is RStudio?

RStudio is an Integrated Development Environment (IDE) for R. It provides a user-friendly interface to write, debug, and execute R code more efficiently. RStudio makes working with R more intuitive by organizing various tools and features into a single workspace.

Key Features of RStudio:

- 1. Source Editor: Write, edit, and save R scripts (`.R`) or R Markdown files (`.Rmd`) in an organized way.
- 2. Console: Interactively run R commands and view outputs.
- 3. Environment/History: View and manage objects (data frames, variables, functions) created during your R session and access command history.
- 4. Plots and Viewer: Visualize your data and access saved plots in one place.
- 5. Packages Pane: Install, update, and load packages directly from the RStudio interface.
- 6. Integration with Git: Perform version control operations for collaborative projects.

Comparing R and RStudio

Aspect	R	RStudio
Definition	A programming language for statistical computing.	An IDE for working with R.

Aspect	R	RStudio
Usage	Can be run independently from a terminal or script.	Provides a graphical interface for R.
User-Friendliness	Requires knowledge of command-line operations.	More intuitive for beginners.
Additional Features	Focused solely on running R code.	Includes debugging, plotting, and file management.

Why Use R and RStudio?

Using R alongside RStudio creates a seamless workflow for data analysis, visualization, and reporting. Here are some examples of how they work together:

1. Efficient Coding: Write and test code in the RStudio source editor while running commands interactively in the console.
2. Data Visualization: Use R packages like `ggplot2` to generate visualizations and preview them in the Plots pane.
3. Reproducible Research: Use R Markdown in RStudio to create reproducible reports that combine text, code, and results.

Getting Started with R and RStudio

Step 1: Installing R

1. Go to the official R website: <https://cran.r-project.org/> (<https://cran.r-project.org/>).
2. Download the version of R suitable for your operating system (Windows, macOS, or Linux).
3. Follow the installation prompts to set up R.

Step 2: Installing RStudio

1. Go to the RStudio website: <https://www.rstudio.com/> (<https://www.rstudio.com/>).
2. Download the free RStudio Desktop version.
3. Install RStudio after installing R (RStudio requires R to function).

Exploring the RStudio Interface

Once you open RStudio, you'll see four main panes:

1. Source Editor (Top-Left):

- Used for writing R scripts or R Markdown files.
- Features syntax highlighting and code completion.

2. Console (Bottom-Left):

- Interactively execute R commands.
- Displays immediate outputs, errors, and warnings.

3. Environment/History (Top-Right):

- Environment Tab: Lists all objects (e.g., variables, datasets) in memory.
- History Tab: Tracks previously executed commands.

4. Files/Plots/Packages/Help/Viewer (Bottom-Right):

- Files: Manage files in your working directory.
 - Plots: View plots created in your R session.
 - Packages: Install, load, or update R packages.
 - Help: Access documentation for R functions and packages.
 - Viewer: Preview web content, such as Shiny apps or R Markdown outputs.
-

R packages and Libraries.

In R, packages and libraries are integral components that help extend the capabilities of the base R environment. They are both essential in enhancing the functionality of R, enabling users to perform complex tasks with ease, without having to reinvent the wheel.

R package.

An R package is a bundled collection of pre-written R functions, datasets, and sometimes compiled code that is designed to extend the functionality of R. Packages are created to solve specific tasks or provide tools within certain domains, such as data visualization, machine learning, or statistical analysis. These packages can be shared with others, making it easier for users to access and utilize code written by others, saving time and effort.

R packages are stored in repositories such as:

- CRAN (Comprehensive R Archive Network): The main repository for R packages, offering a vast collection of user-contributed packages.
- Bioconductor: A repository specializing in bioinformatics packages, particularly for the analysis of genomic data.
- GitHub: A platform where developers can share R packages (sometimes in development or not yet officially published to CRAN).

Examples of Popular R Packages:

- ggplot2: A powerful package for creating complex and customizable data visualizations.
- dplyr: A popular package for data manipulation, providing an intuitive set of tools for transforming and summarizing data.
- tidyr: A package designed to help tidy and reshape data, making it easier to work with in R.

- **caret**: A comprehensive package for building machine learning models, offering tools for data preprocessing, training, and model evaluation.

To access any package in, it should be installed at first (some of the packages are already loaded in R, but their number is quite less) To install the packages, the function, `install.packages()` is used. You only need to install a package once on your system.

To exemplify, if we want to install `ggplot2` package we write it as `install.packages("ggplot2")` (Remove the `#` before installing the package as `#` converts anything written behind it into a comment.)To use a package in R, it first needs to be installed. While R comes with a few pre-installed packages, the number of these is relatively small. Most packages need to be installed from external repositories like CRAN, Bioconductor, or GitHub.

To install a package, you use the `install.packages()` function. Here's how you would install the `ggplot2` package:

```
#install.packages("ggplot2")
```

R libraries

In R, a library refers to the directory or location on your computer where installed packages are stored. It's where R looks to find the packages you want to use. When you install a package, it is saved in your library, and you can then load it into your R session for use.

To access the functions and datasets within a package, you need to load it into your R environment using the `library()` function. For example:

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

This command makes all of the functions and datasets from the `ggplot2` package available for use in your current session. But if you start the other session, you need to fetch the package again.